



TRAVAUX DIRIGÉS – SÉRIE 3

Algorithmique et Programmation

Simulation, théorie des jeux, géométrie et structures de données

Ce document regroupe une troisième série de problèmes algorithmiques issus de compétitions de programmation.

1 Snapper Chain

Exercice 1 — Snapper Chain

Un *Snapper* est un petit appareil doté d'une prise d'entrée et d'une prise de sortie. Lorsqu'un Snapper est en état **ON** et reçoit du courant, il transmet le courant à l'appareil branché sur sa sortie. Quand vous claquez des doigts, tout Snapper recevant du courant **bascule** entre **ON** et **OFF**.

Vous avez acheté N Snappers et les avez branchés en chaîne : le premier dans une prise murale, le deuxième dans le premier, etc. Une lampe est branchée dans le N -ième Snapper. Initialement, tous les Snappers sont **OFF**.

Objectif : Après K claquements de doigts, la lampe est-elle allumée (**ON**) ou éteinte (**OFF**) ? La lampe est **ON** si et seulement si elle reçoit du courant.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : deux entiers N et K ($1 \leq N, K \leq 10^8$).

Sortie : **ON** ou **OFF** pour chaque cas.

2 Theme Park

Exercice 2 — Theme Park

Un roller coaster a une capacité de k personnes et tourne R fois par jour. Les visiteurs font la queue par groupes. Les groupes montent dans l'ordre : on ajoute les groupes un par un tant qu'il reste de la place (un groupe ne monte que s'il rentre entièrement). Après le tour, les passagers se remettent dans la queue dans le même ordre.

Objectif : Calculer le revenu total de la journée (1 euro par personne par tour).

Exemple : $R = 4$, $k = 6$, groupes de tailles $(1, 4, 2, 1)$.

- Tour 1 : groupes $[1, 4]$ montent (5 personnes). Queue : $(2, 1, 1, 4)$.
- Tour 2 : $[2, 1, 1]$ montent (4 personnes). Queue : $(4, 2, 1, 1)$.
- Tour 3 : $[4, 2]$ montent (6 personnes). Queue : $(1, 1, 4, 2)$.
- Tour 4 : $[1, 1, 4]$ montent (6 personnes). Total : $5 + 4 + 6 + 6 = 21$ euros.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : R , k , N , puis N tailles de groupes. R peut aller jusqu'à 10^8 , $N \leq 1000$.

Sortie : Le revenu total.

3 Fair Warning

Exercice 3 — Fair Warning

N grands événements se sont produits respectivement $t_1, t_2, \dots, t_N$ slarbosecondes avant le moment présent. L'apocalypse arrivera dans $y \geq 0$ slarbosecondes, au moment où chaque durée $(t_i + y)$ sera divisible par un même nombre T , le plus grand possible.

Objectif : Trouver la plus petite valeur de $y \geq 0$ telle que $T = \gcd(t_1 + y, t_2 + y, \dots, t_N + y)$ soit maximal.

Exemple : Événements à 26000, 11000, 6000 slarbosecondes. Avec $y = 4000$, les durées deviennent 30000, 15000, 10000, et $\gcd = 5000$.

Format d'entrée / sortie

Entrée : T_c cas. Chaque cas : N , puis N entiers (potentiellement très grands).

Sortie : La valeur minimale de y .

4 Rotate

Exercice 4 — Rotate (Join-K)

Dans le jeu Join- K , des pièces rouges (R) et bleues (B) sont empilées dans une grille $N \times N$ avec gravité vers le bas. Un joueur gagne s'il aligne K pièces de sa couleur (horizontalement, verticalement ou en diagonale).

Vous décidez de tricher en **tournant le plateau de 90° dans le sens horaire**. Après rotation, la gravité fait retomber les pièces.

Objectif : Déterminer quel joueur (Rouge, Bleu, les deux, ou aucun) a K pièces alignées après la rotation et l'application de la gravité.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N , K , puis la grille $N \times N$ ($.$ = vide, R = rouge, B = bleu).

Sortie : Red, Blue, Both ou Neither.

5 File Fix-it

Exercice 5 — File Fix-it

Sur un système Unix, les données sont stockées dans des répertoires organisés en arbre. La commande `mkdir` crée un répertoire, mais **uniquement si le répertoire parent existe déjà**.

Objectif : Étant donné l'ensemble des répertoires existants et une liste de répertoires à créer (s'ils n'existent pas encore), déterminer le nombre **minimum** de commandes `mkdir` nécessaires.

Pour créer `/home/gcj/finals` et `/home/gcj/quals` depuis un système vide, il faut 4 commandes : `mkdir /home`, `mkdir /home/gcj`, `mkdir /home/gcj/finals`, `mkdir /home/gcj/quals`.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N (répertoires existants) et M (répertoires à créer), puis les chemins.

Sortie : Le nombre de `mkdir` nécessaires.

6 Picking Up Chicks

Exercice 6 — Picking Up Chicks

N poussins courent vers l'est sur une route rectiligne vers une grange située en position B . Le poussin i part de la position X_i avec une vitesse V_i . Si un poussin rattrape celui qui le précède, il ralentit et suit à sa vitesse.

Vous disposez d'une grue mobile qui peut **échanger** deux poussins adjacents instantanément (le poussin arrière laisse passer celui de devant).

Objectif : Quel est le nombre **minimum** d'échanges pour qu'au moins K poussins arrivent à la grange au plus tard au temps T ? Si c'est impossible, indiquer **IMPOSSIBLE**.

Format d'entrée / sortie

Entrée : T_c cas. Chaque cas : N, K, B, T , puis les positions et vitesses des poussins.

Sortie : Le nombre minimal d'échanges ou **IMPOSSIBLE**.

7 Rope Intranet

Exercice 7 — Rope Intranet

Une entreprise occupe deux immeubles très hauts. L'intranet est constitué de câbles reliant des fenêtres d'un immeuble à des fenêtres de l'autre. Chaque fenêtre a au plus un câble. Vu de côté, les câbles sont des segments droits.

Objectif : Compter le nombre de **points d'intersection** entre les câbles (exactement deux câbles se croisent en chaque point d'intersection).

Deux câbles (a_1, b_1) et (a_2, b_2) se croisent si et seulement si $(a_1 - a_2)$ et $(b_1 - b_2)$ sont de signes opposés.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N câbles, puis N paires (a_i, b_i) = hauteurs des fenêtres gauche et droite.

Sortie : Le nombre d'intersections.

8 Load Testing

Exercice 8 — Load Testing

Un site web peut supporter au moins L utilisateurs simultanés, mais pas P utilisateurs. Vous souhaitez connaître sa capacité à un facteur C près : trouver un entier a tel que le site supporte a utilisateurs mais pas $a \times C$.

Chaque *load test* vous indique si le site supporte au moins X utilisateurs (pour un X que vous choisissez). Vous pouvez adapter votre stratégie en fonction des résultats précédents.

Objectif : Dans le **pire cas**, combien de load tests sont nécessaires avec une stratégie optimale ?

Format d'entrée / sortie

Entrée : T cas. Chaque cas : trois entiers L, P, C ($2 \leq C, 1 \leq L < P \leq 10^9$).

Sortie : Le nombre minimal de tests dans le pire cas.

9 Hot Dog Prolifération

Exercice 9 — Hot Dog Prolifération

Des vendeurs de hot-dogs sont installés à des intersections le long d'une rue infinie (axe est-ouest). Certaines intersections ont plusieurs vendeurs.

Un **mouvement** consiste à choisir une intersection avec ≥ 2 vendeurs : l'un se déplace d'une intersection vers l'est, l'autre d'une vers l'ouest.

Objectif : Trouver le nombre **minimum** de mouvements pour que tous les vendeurs soient à des intersections différentes.

Configuration initiale : ...0 0 2 1 2 0 0...

Après 3 mouvements : ...0 1 1 1 1 1 0...

Format d'entrée / sortie

Entrée : T cas. Chaque cas : C (nombre d'intersections occupées), puis C paires (position, nombre de vendeurs).

Sortie : Le nombre minimal de mouvements.

10 Fence

Exercice 10 — Fence

On veut construire une clôture de longueur exactement L . On peut acheter des planches en quantités illimitées, parmi N longueurs disponibles $B_1, B_2, \dots, B_N$.

Objectif : Trouver le nombre **minimum** de planches à acheter pour obtenir une longueur totale **exactement** égale à L . Attention : L peut être très grand.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : L et N , puis N longueurs de planches. L peut atteindre 10^{18} .

Sortie : Le nombre minimal de planches, ou IMPOSSIBLE.

11 Different Sum

Exercice 11 — Different Sum (Cryptarithmes)

Une **équation cryptarithmétique** est une addition posée verticalement (les nombres alignés à droite) telle que dans chaque colonne, tous les chiffres des *sommandes* sont **distincts** (le chiffre de la somme n'est pas concerné par cette contrainte).

Les sommandes sont positifs, sans zéros de tête. L'ordre des sommandes n'importe pas.

Objectif : Étant donné une somme N en base B , compter le nombre d'équations cryptarithmétiques dont la somme vaut N . Résultat modulo $10^9 + 7$.

En base 10 : $124 + 31 + 25 = 180$ n'est **pas** valide (colonne 2 a deux fois le chiffre 2). Mais $124 + 31 + 15 = 170$ serait valide.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : B (base) et N (la somme, en base B).

Sortie : Le nombre d'équations modulo $10^9 + 7$.

12 Bacteria

Exercice 12 — Bacteria

Des bactéries vivent sur une grille infinie (une par cellule). Chaque seconde, simultanément :

1. Une bactérie qui n'a **aucun voisin** au nord **ni** à l'ouest **meurt**.
2. Une cellule vide dont les voisins au nord **et** à l'ouest contiennent tous deux une bactérie **donne naissance** à une nouvelle bactérie.

Les bactéries initiales forment une ou plusieurs **régions rectangulaires**.

Objectif : Déterminer après combien de secondes toutes les bactéries auront disparu.

Format d'entrée / sortie

Entrée : T cas. Chaque cas décrit les rectangles initiaux de bactéries.

Sortie : Le nombre de secondes avant disparition complète.

13 Elegant Diamond

Exercice 13 — Elegant Diamond

Un **diamant de taille k** est une figure en losange de $2k - 1$ lignes composée de chiffres. Un diamant est **élégant** s'il est symétrique à la fois horizontalement et verticalement. On vous donne un diamant (pas nécessairement élégant). Vous pouvez l'**agrandir** en l'insérant dans un diamant plus grand et en ajoutant des chiffres autour. Le **coût** est le nombre de chiffres ajoutés.

Objectif : Trouver le coût **minimum** pour transformer le diamant donné en un diamant **élégant** (de taille $\geq k$).

Format d'entrée / sortie

Entrée : T cas. Chaque cas : la taille k puis le diamant.

Sortie : Le coût minimal d'agrandissement.

14 Grazing Google Goats

Exercice 14 — Grazing Google Goats

Le fermier John a N chèvres, chacune attachée à un poteau en position P_i par une corde de longueur L_i . Chaque chèvre peut brouter dans un disque de rayon L_i centré sur P_i . Le fermier hésite entre M positions pour placer un seau d'eau Q_j . Il veut que chaque chèvre puisse atteindre le seau, donc la corde de la chèvre i doit être au moins $\|P_i - Q_j\|$. Pour chaque position Q_j , choisir les longueurs de corde **minimales** et calculer l'aire A_j de la zone accessible par **toutes** les chèvres (intersection des disques).

Objectif : Calculer A_j pour chaque position de seau.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : N, M , positions des poteaux et des seaux.

Sortie : Les aires $A_1, A_2, \dots, A_M$ avec au moins 6 décimales.

15 World Cup 2010

Exercice 15 — World Cup 2010

Un tournoi à élimination directe a 2^P équipes (identifiées de 0 à $2^P - 1$). Les matchs sont déterminés en appariant les deux équipes restantes de plus petits identifiants. Chaque match a un prix de billet donné.

Varva veut assister au tournoi. Pour chaque équipe i , il accepte de manquer au plus $M[i]$ de ses matchs. Il doit acheter ses billets **à l'avance** (avant de connaître les résultats) et doit garantir le respect de ses contraintes **quel que soit** le déroulement du tournoi.

Objectif : Trouver le coût **minimum** des billets à acheter.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : P , les prix des matchs, et le tableau M .**Sortie :** Le coût minimal.

16 Making Chess Boards

Exercice 16 — Making Chess Boards

Une feuille rectangulaire d'écorce est une grille de cases noires et blanches. Vous devez y découper des **échiquiers** : des carrés dont les cases forment un motif d'échiquier (aucune paire de cases adjacentes de même couleur).

Règle : À chaque étape, découper le **plus grand** échiquier possible. En cas d'égalité, choisir le plus haut puis le plus à gauche. Répéter jusqu'à épuisement de l'écorce (des échiquiers 1×1 comptent).

Objectif : Lister tous les échiquiers découpés avec leur taille et leur nombre.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : R , C , puis la grille en hexadécimal.**Sortie :** Pour chaque taille découpée (ordre décroissant), la taille et le nombre d'échiquiers.

17 Your Rank is Pure

Exercice 17 — Your Rank is Pure

Soit S un sous-ensemble de $\{2, 3, \dots, n\}$. Le **rang** d'un élément x dans S est sa position dans S trié. Un nombre n est dit **pur** par rapport à S si, en partant de n et en prenant itérativement le rang dans S , on finit par atteindre 1 (qui n'est pas dans S).

Exemple : 127 est pur car $127 \rightarrow 31^{\text{e}} \text{ premier} \rightarrow 11 \rightarrow 5 \rightarrow 3 \rightarrow 2$ (la chaîne de rangs parmi les premiers atteint 1).

Objectif : Étant donné n , compter le nombre de sous-ensembles $S \subseteq \{2, \dots, n\}$ tels que n soit pur par rapport à S . Résultat modulo 100 003.

Format d'entrée / sortie**Entrée :** T cas. Chaque cas : un entier n .**Sortie :** Le nombre de sous-ensembles modulo 100 003.

18 Make it Smooth

Exercice 18 — Make it Smooth

On dispose d'un tableau 1D de N pixels, chacun ayant une valeur entre 0 et 255. La **distance** entre deux pixels est la différence absolue de leurs valeurs. Le tableau est **lisse** si toute paire de pixels voisins a une distance $\leq M$.

Opérations disponibles (coûts) :

1. **Supprimer** un pixel : coût D (ses voisins deviennent adjacents).
2. **Insérer** un pixel de valeur quelconque : coût I .
3. **Modifier** la valeur d'un pixel : coût = $|\text{ancienne} - \text{nouvelle}|$.

Objectif : Trouver le coût **minimum** pour rendre le tableau lisse (le tableau vide est considéré lisse).

Format d'entrée / sortie

Entrée : T cas. Chaque cas : D, I, M, N , puis N valeurs de pixels.

Sortie : Le coût minimal.

19 Number Game

Exercice 19 — Number Game

Arya et Bran jouent un jeu. Deux entiers positifs A et B sont écrits au tableau. Les joueurs alternent (Arya commence). À chaque tour, un joueur peut :

- Remplacer A par $A - k \cdot B$ pour un entier $k > 0$, ou
- Remplacer B par $B - k \cdot A$ pour un entier $k > 0$.

Le premier joueur qui fait tomber un nombre à zéro ou en dessous **perd**.

Objectif : Étant donné A_1, A_2, B_1, B_2 , compter le nombre de paires (A, B) avec $A_1 \leq A \leq A_2$ et $B_1 \leq B \leq B_2$ qui sont des **positions gagnantes** pour Arya.

Avec $(A, B) = (12, 51)$: Arya remplace 51 par $51 - 3 \times 12 = 15 \rightarrow (12, 15)$. Le jeu continue. L'analyse est liée à l'algorithme d'Euclide.

Format d'entrée / sortie

Entrée : T cas. Chaque cas : A_1, A_2, B_1, B_2 .

Sortie : Le nombre de positions gagnantes.

Bon courage et bonne programmation !